

SplitX

A mobile-first group-expense platform that reduces any debt graph to the minimum number of settlements — delivered on a production DevOps pipeline: containerised, auto-scaled on Kubernetes, provisioned on AWS as code, and fully observed.

NEXT.JS 16 · REACT 19

DOCKER

KUBERNETES + HPA

TERRAFORM · AWS EKS

CLOUDFRONT

JENKINS · GITHUB ACTIONS

PROMETHEUS · GRAFANA · LOKI

P Problem Statement

Shared spending produces a dense debt graph: with **n** participants each expense creates up to **n-1** obligations, so naive settlement needs **O(n²)** transfers. Manual tracking is error-prone, itemised bills cannot be divided fairly by hand, and cash reconciliation leaves no audit trail. Equally, a feature-complete application without a reproducible build, an automated release path, a scaling policy or operational visibility **cannot be run in production** — deployments become manual, failures silent and capacity guesswork.

S Proposed Solution

SplitX records expenses under **four split strategies** and collapses the debt graph to the **minimum transfer set** using a dual-algorithm settlement engine (greedy netting benchmarked against exact-match pruning). Receipts are itemised by OCR and vision AI, expenses can be dictated by voice, and settlement hands off to UPI. The platform is delivered by an **end-to-end DevOps pipeline**: multi-stage container image, Kubernetes orchestration with horizontal pod autoscaling, Terraform-provisioned AWS infrastructure behind CloudFront, gated Jenkins and GitHub Actions pipelines, and a metrics + logs + alerting stack.

O Objectives

- Model group expenses and compute the **minimum settlement set** with an immutable audit trail.
- Package the application as a **reproducible, non-root, minimal-surface** container image.
- Provision the complete **AWS footprint declaratively** with Terraform and remote state.
- Orchestrate on Kubernetes with **self-healing, zero-downtime rollouts and CPU-driven HPA**.
- Automate build → test → scan → publish → deploy on **webhook triggers**, with gates that can fail the build.
- Instrument **metrics, dashboards, centralised logs and alert routing** end to end.

C Scope & Delimitations

- **In scope:** the web application and its container image; Kubernetes manifests and Helm chart; Terraform modules for VPC, EKS, ECR, S3 and CloudFront; Jenkins and GitHub Actions pipelines; Prometheus, Alertmanager, Grafana, Loki; GitOps reconciliation with ArgoCD.
- **Out of scope:** native iOS/Android builds (the PWA is installable instead); multi-currency FX conversion; and custody of funds — UPI deep links delegate the actual transfer to the user's own payment app, so SplitX never touches money.
- **Assumptions:** a managed PostgreSQL endpoint is reachable; AWS resources are created for the demonstration window and destroyed afterwards to contain cost.

TECHNOLOGY STACK

Application

Next.js 16 (App Router) React 19 TypeScript 5 Prisma 6 NextAuth v5 Zod SWR Framer Motion Recharts
CSS Modules PWA / Workbox

Data & AI

Neon PostgreSQL Upstash Redis Supabase Storage Gemini 2.0 Flash GPT-4o-mini Vision Tesseract.js
Web Speech API Resend

Containers, Cloud & IaC

Docker Docker Compose Kubernetes Helm Kustomize ArgoCD AWS EKS Amazon ECR CloudFront
Amazon S3 VPC / ALB Terraform Ansible

CI/CD & Observability

Git + GitHub GitHub Actions Jenkins SonarQube Trivy Nexus ESLint Husky Commitlint Prometheus
Alertmanager Grafana Loki Promtail

18

Prisma data models

38

API route handlers

19

DevOps tools integrated

12

Jenkins pipeline stages

2→10

Pods via HPA

3-stage

Docker image build

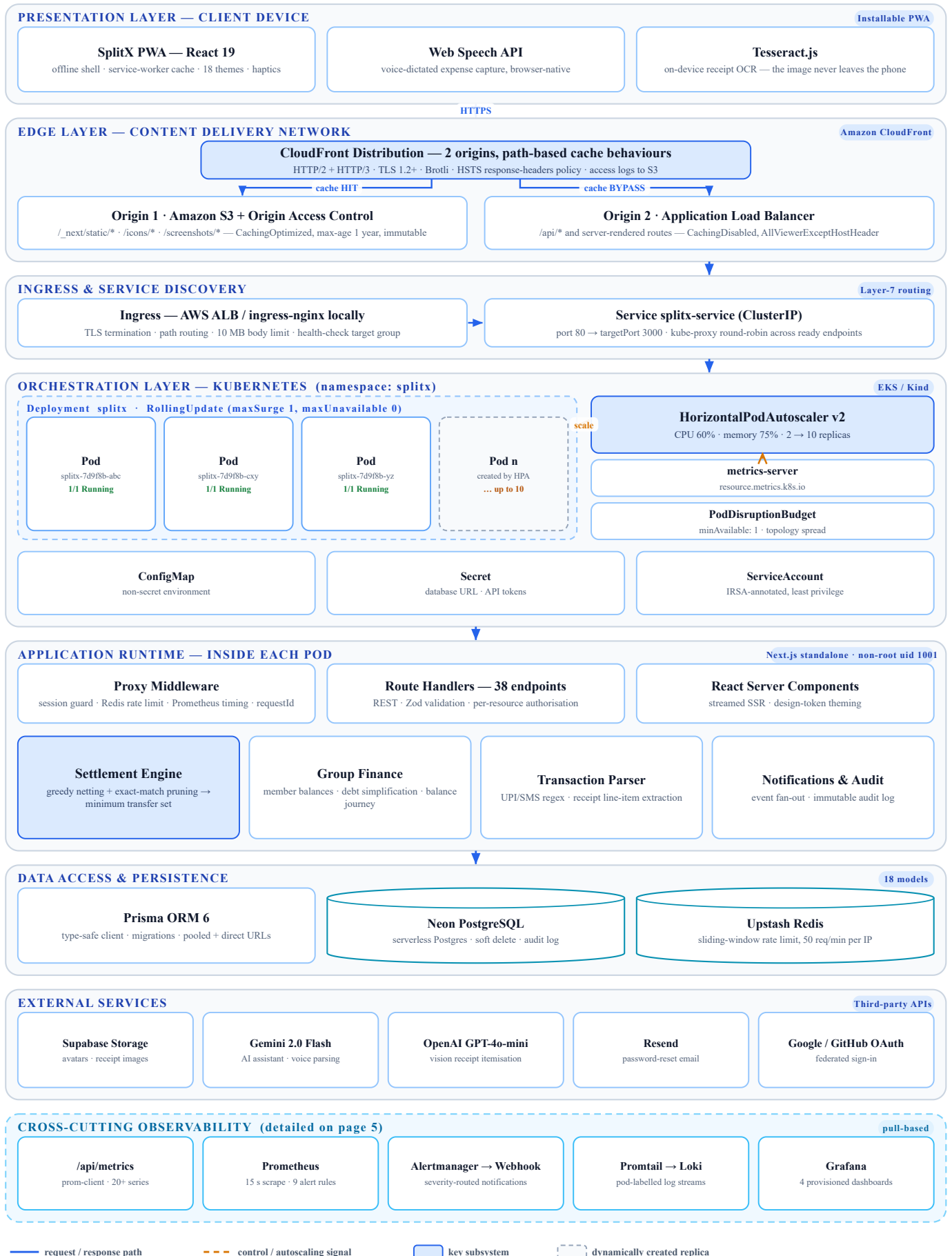


Figure 1 — Layered system architecture: client → CDN → ingress → Kubernetes → application runtime → data → external services, with the cross-cutting observability plane.

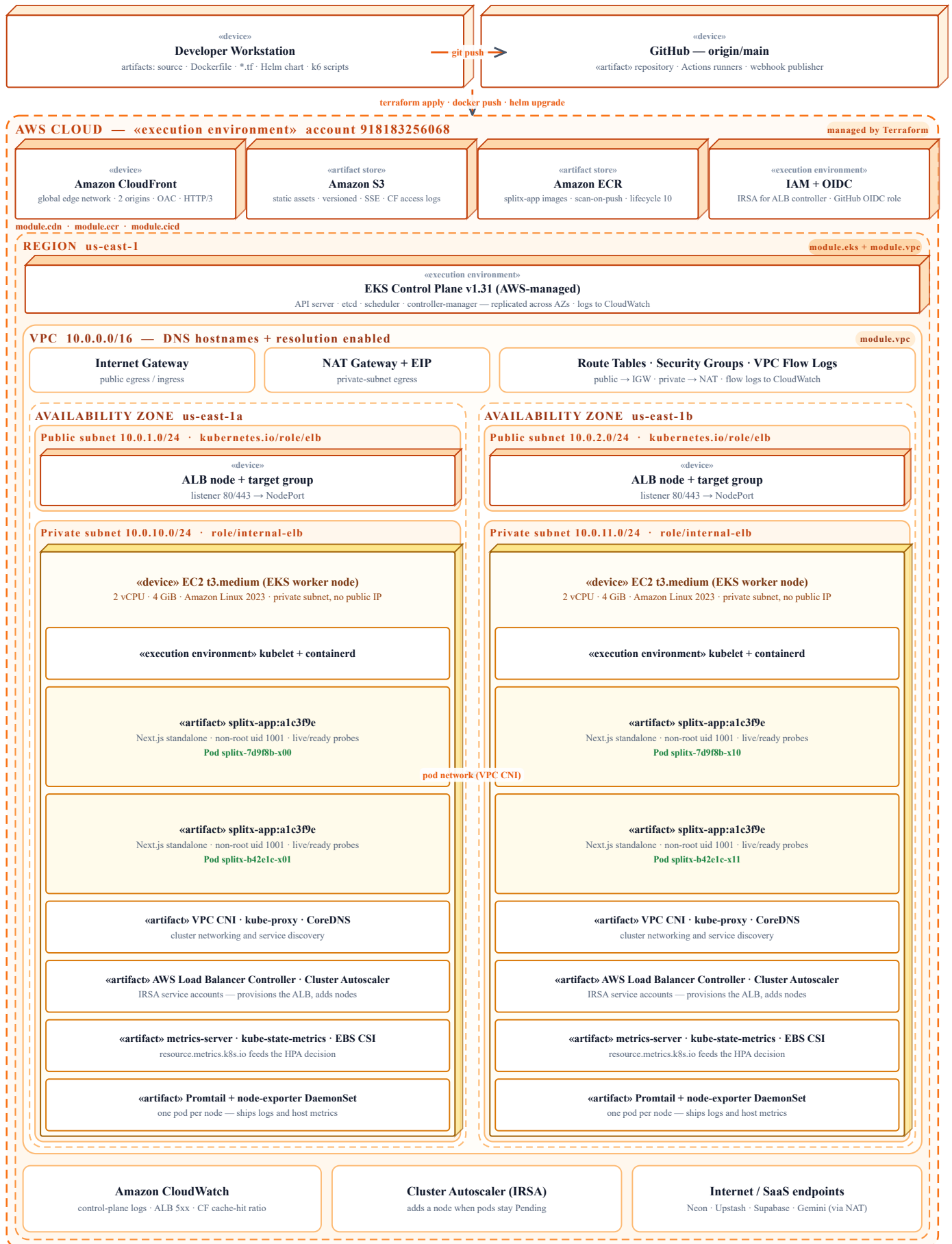


Figure 2 — UML deployment diagram of the Terraform-provisioned AWS footprint: «device» and «execution environment» nodes with the «artifact»s deployed onto them, annotated with the owning Terraform module.

Continuous Integration & Delivery Pipeline JENKINS · ON-PREMISES

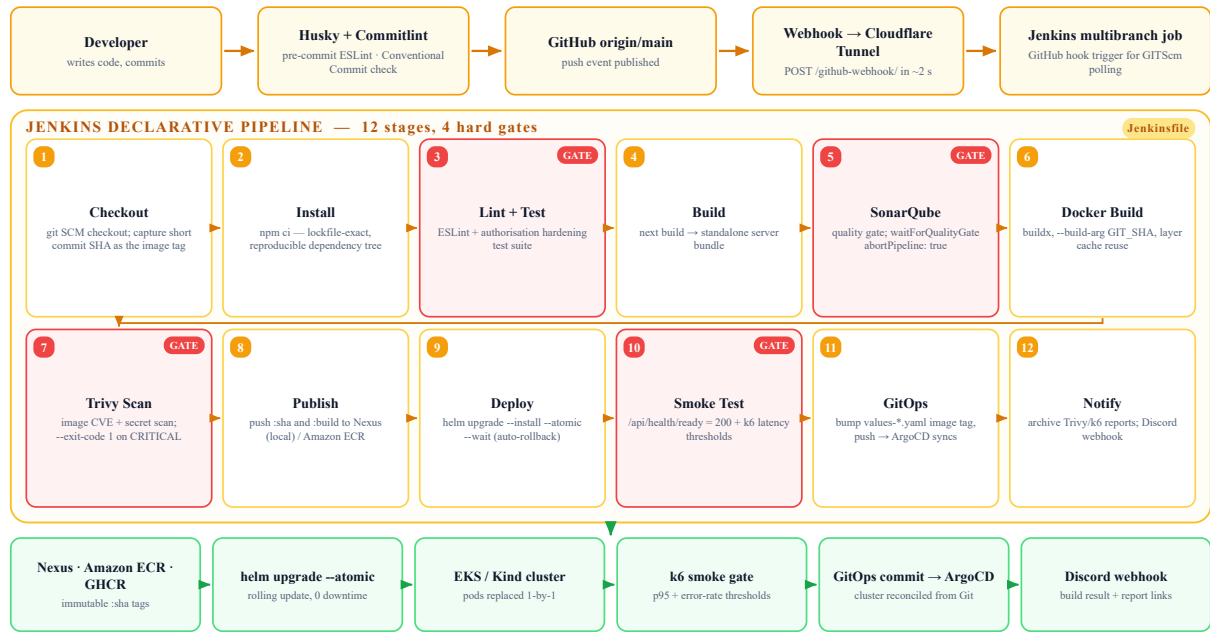


Figure 3 — Trigger chain, the twelve Jenkins stages with their four build-failing gates, and the delivery outputs.

Cloud CI/CD & Keyless AWS Access GITHUB ACTIONS



Figure 4 — The four GitHub Actions workflows and the OIDC trust path that removes long-lived AWS credentials entirely.

Toolchain Responsibilities TABLE 1

TOOL · CATEGORY	CONCRETE ROLE IN SPLITX	TOOL · CATEGORY	CONCRETE ROLE IN SPLITX
Git + GitHub SCM	Single source of truth; PR flow, protected main, signed history	Nexus ARTIFACT REGISTRY	Private Docker registry for local pipeline runs
Husky HOOKS	Client-side git hooks — lints and blocks a bad commit before it exists	Terraform IAC	Whole AWS footprint as code; S3 remote state with locking
Commitlint STANDARDS	Enforces Conventional Commits so history is machine-readable	Ansible CONFIG MGMT	Idempotent host configuration of worker nodes post-provision
ESLint STATIC ANALYSIS	Type-aware lint of TS/React; a violation fails the pipeline	Kubernetes ORCHESTRATION	Self-healing, rolling updates, probes, quotas, HPA
GitHub Actions CLOUD CI	Matrix build, GHCR publish, Trivy SARIF, keyless OIDC deploy	Helm PACKAGING	Templated versioned releases; atomic upgrade, one-command rollback
Docker CONTAINERISATION	3-stage build → non-root, minimal-surface runtime image	ArgoCD GITOPS CD	Continuously reconciles live cluster state against Git
Docker Compose LOCAL STACK	10 services: app, Postgres, Redis, monitoring, CI tools	Prometheus + Alertmanager METRICS + ALERTING	Pull scraping, recording rules, 9 alert rules, webhook routing
Jenkins CI ORCHESTRATOR	12-stage declarative pipeline with four build-failing gates	Grafana VISUALISATION	4 provisioned dashboards; metric ↔ log correlation
SonarQube QUALITY GATE	Bugs, smells, duplication; aborts the pipeline on failure	Loki + Promtail LOG AGGREGATION	Label-indexed centralised logs queried with LogQL
Trivy SECURITY	Image CVE + secret scan; non-zero exit on CRITICAL findings		

Observability Data Flow METRICS · LOGS · ALERTS

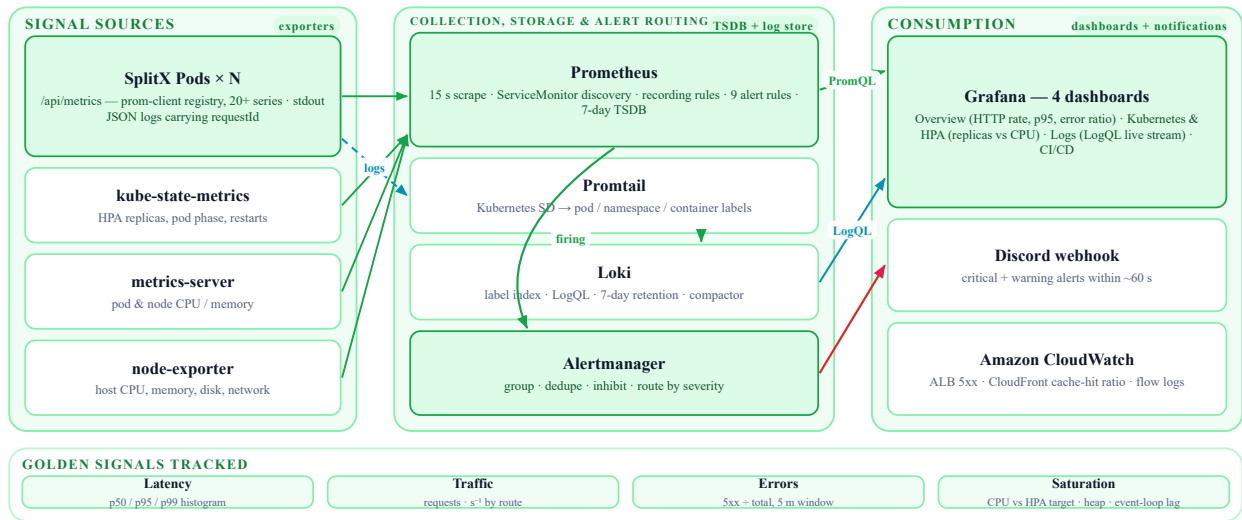
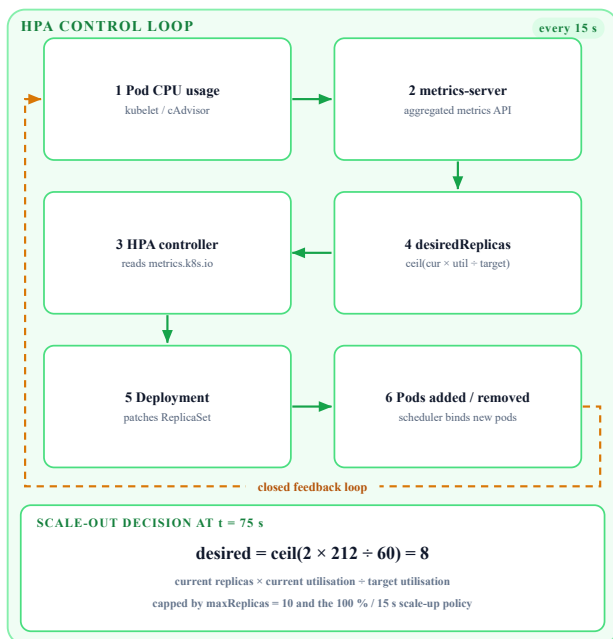


Figure 5 — Signal sources, collection and storage, and the consumption surfaces; the four golden signals tracked throughout.

Horizontal Pod Autoscaling — Mechanism and Measured Behaviour KUBERNETES HPA V2



```
$ kubectl get hpa splitx-hpa -n splitx -w
NAME          REFERENCE    TARGETS      MIN MAX  REP
splitx-hpa    Deploy/splitx  cpu: 4%/60%   2  10  2
splitx-hpa    Deploy/splitx  cpu: 212%/60%  2  10  2
splitx-hpa    Deploy/splitx  cpu: 212%/60%  2  10  4
splitx-hpa    Deploy/splitx  cpu: 141%/60%  2  10  6
splitx-hpa    Deploy/splitx  cpu: 106%/60%  2  10  8
splitx-hpa    Deploy/splitx  cpu: 79%/60%   2  10  10
splitx-hpa    Deploy/splitx  cpu: 55%/60%   2  10  10
splitx-hpa    Deploy/splitx  cpu: 4%/60%    2  10  2

$ kubectl get pods -n splitx -o wide
NAME          READY  STATUS   NODE
splitx-7d9f8b-abc  1/1    Running  worker-1
splitx-7d9f8b-cxy  1/1    Running  worker-2
splitx-7d9f8b-x00  1/1    Running  worker-1
splitx-7d9f8b-yz1  0/1    Pending  <none>

# topologySpread keeps replicas even across nodes

$ k6 run loadtest/k6/ramp.js
checks.....: 100.00% ✓ 41812 × 0
http_req_duration: avg=118ms p(95)=412ms
http_req_failed..: 0.00% ✓ 0 × 41812
iterations.....: 41812 174.2/s
# 0 failed requests during the rollout
```

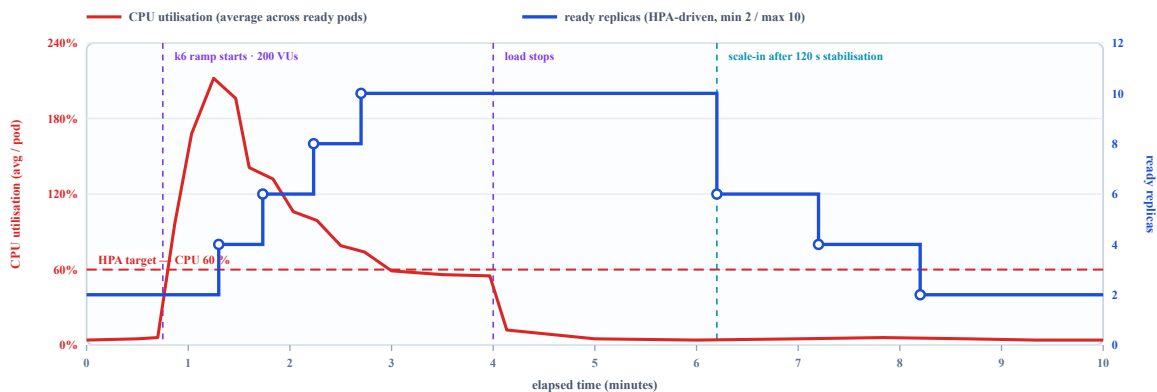


Figure 6 — The HPA reconciliation loop (left) with live kubectl and k6 output (right). Figure 7 — CPU utilisation against ready replicas during a k6 ramp: scale-out 2→10, then scale-in after the 120 s stabilisation window.

Use Case Diagram UML BEHAVIOURAL

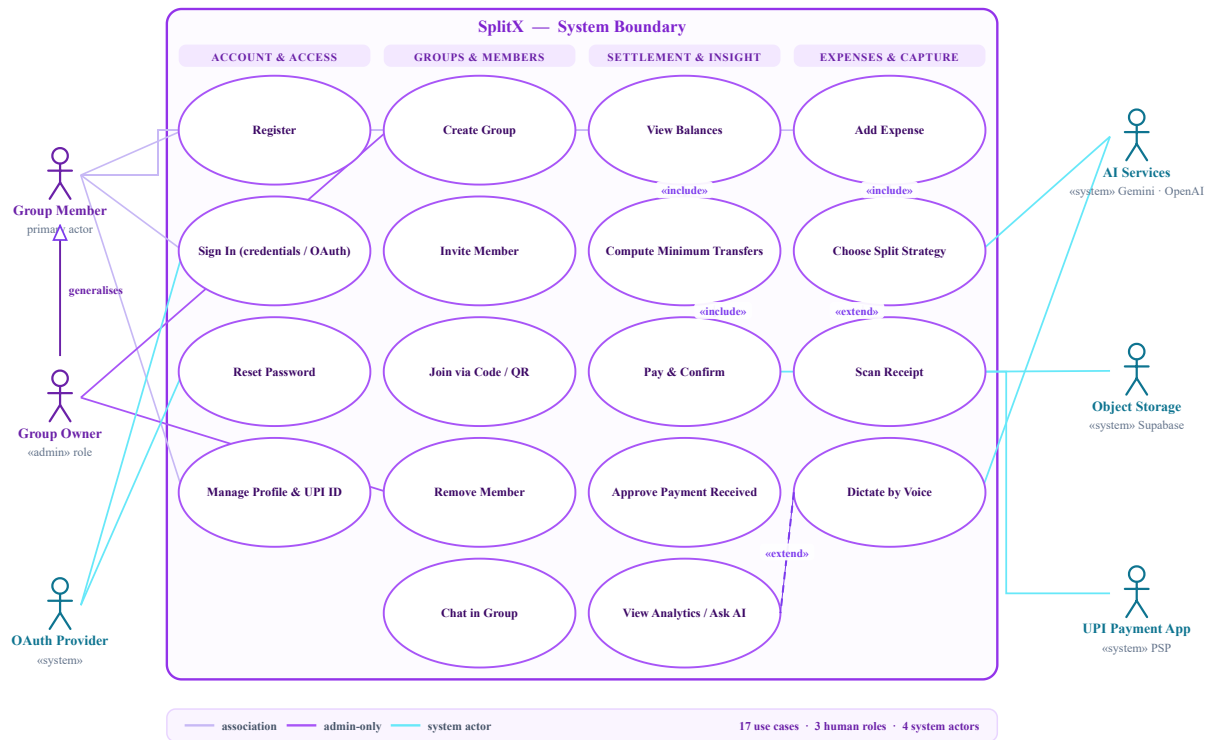


Figure 8 — Seventeen use cases grouped into four functional packages, with three human roles, four system actors, and «include»/«extend» relationships.

Activity Diagram — Record an Expense and Settle UML BEHAVIOURAL

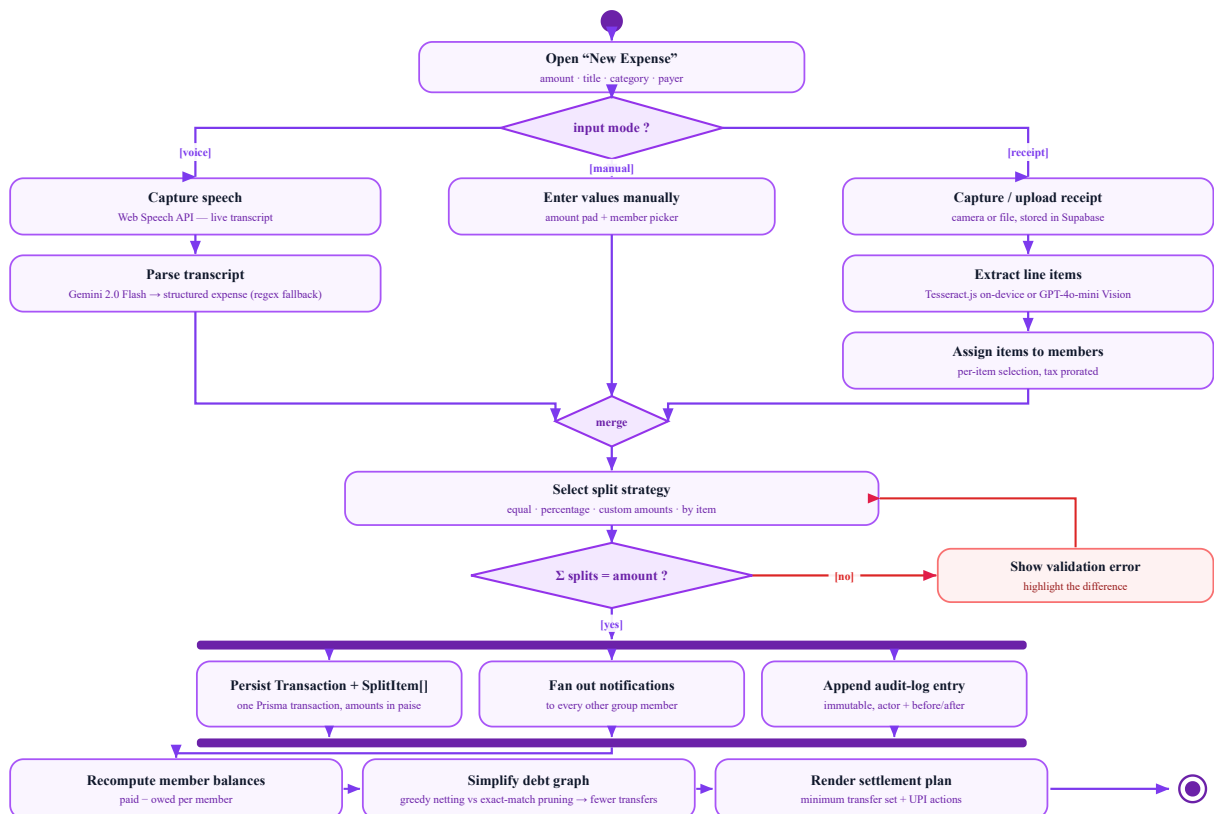
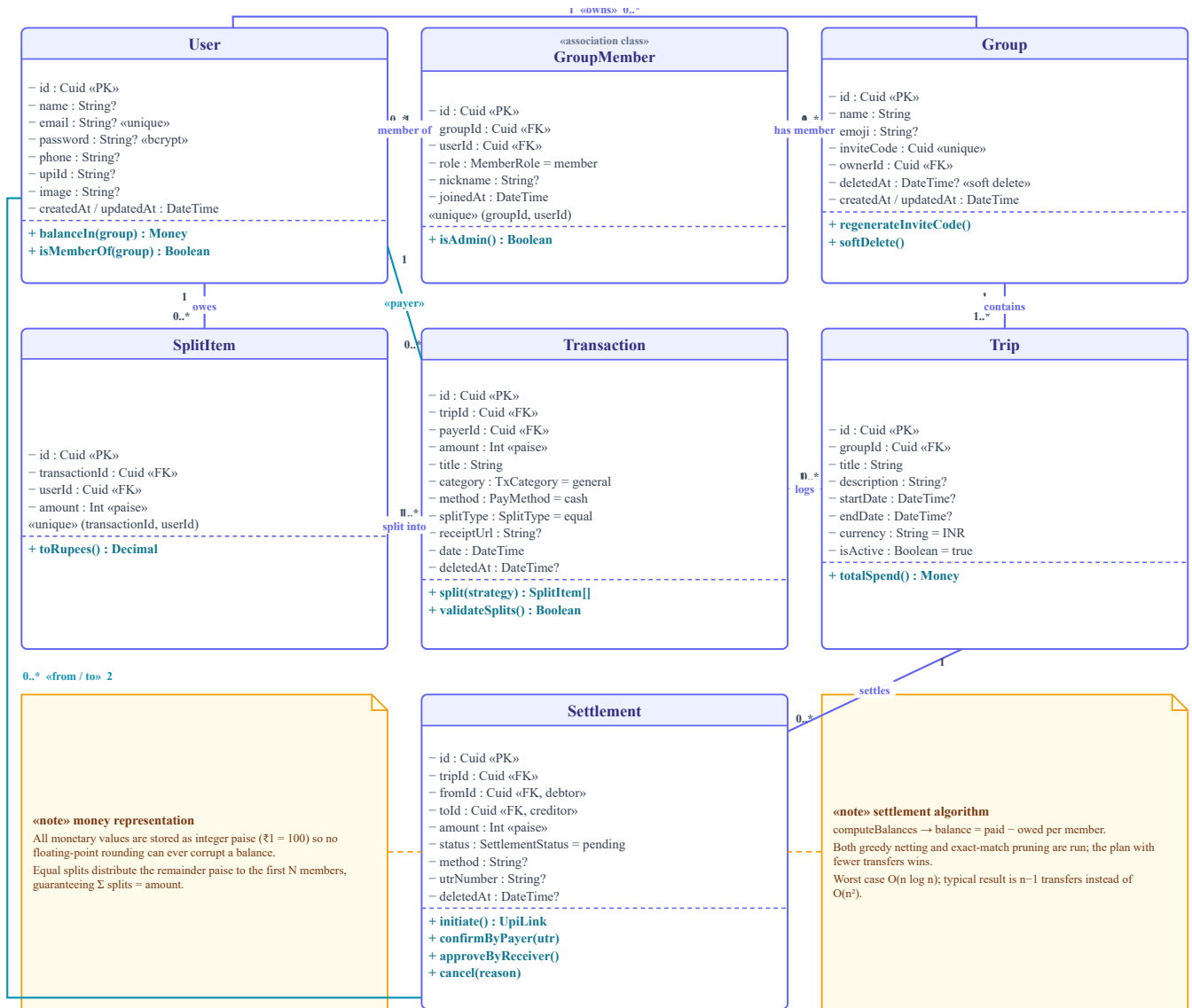
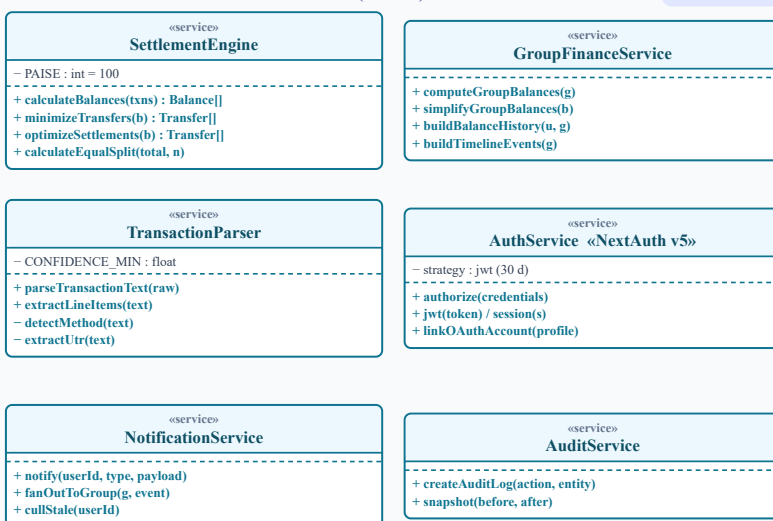


Figure 9 — End-to-end activity flow with a three-way input decision, a validation loop, and a fork/join for the parallel persistence, notification and audit actions.



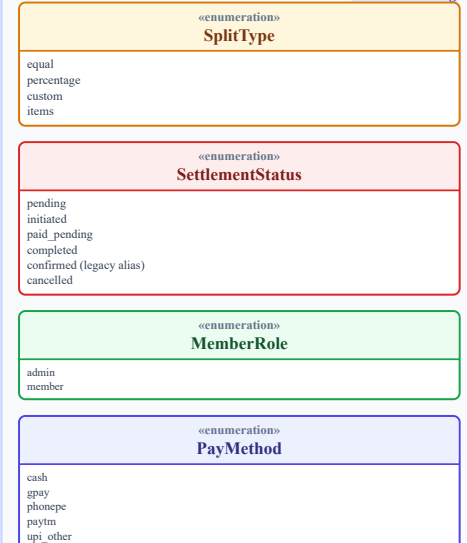
SERVICE / DOMAIN-LOGIC CLASSES (src/lib)

stateless modules



ENUMERATED ATTRIBUTE DOMAINS

stored as String



— association + multiplicity ◆ composition (whole ◆ part) - - - «include» / «extend» / note anchor «PK» primary key «FK» foreign key «unique» constraint

Figure 10 — Core domain classes with attributes, operations, multiplicities and a composition; the stateless service classes that implement the business rules; and the enumerated attribute domains.

Entity-Relationship Diagram LOGICAL DATA MODEL

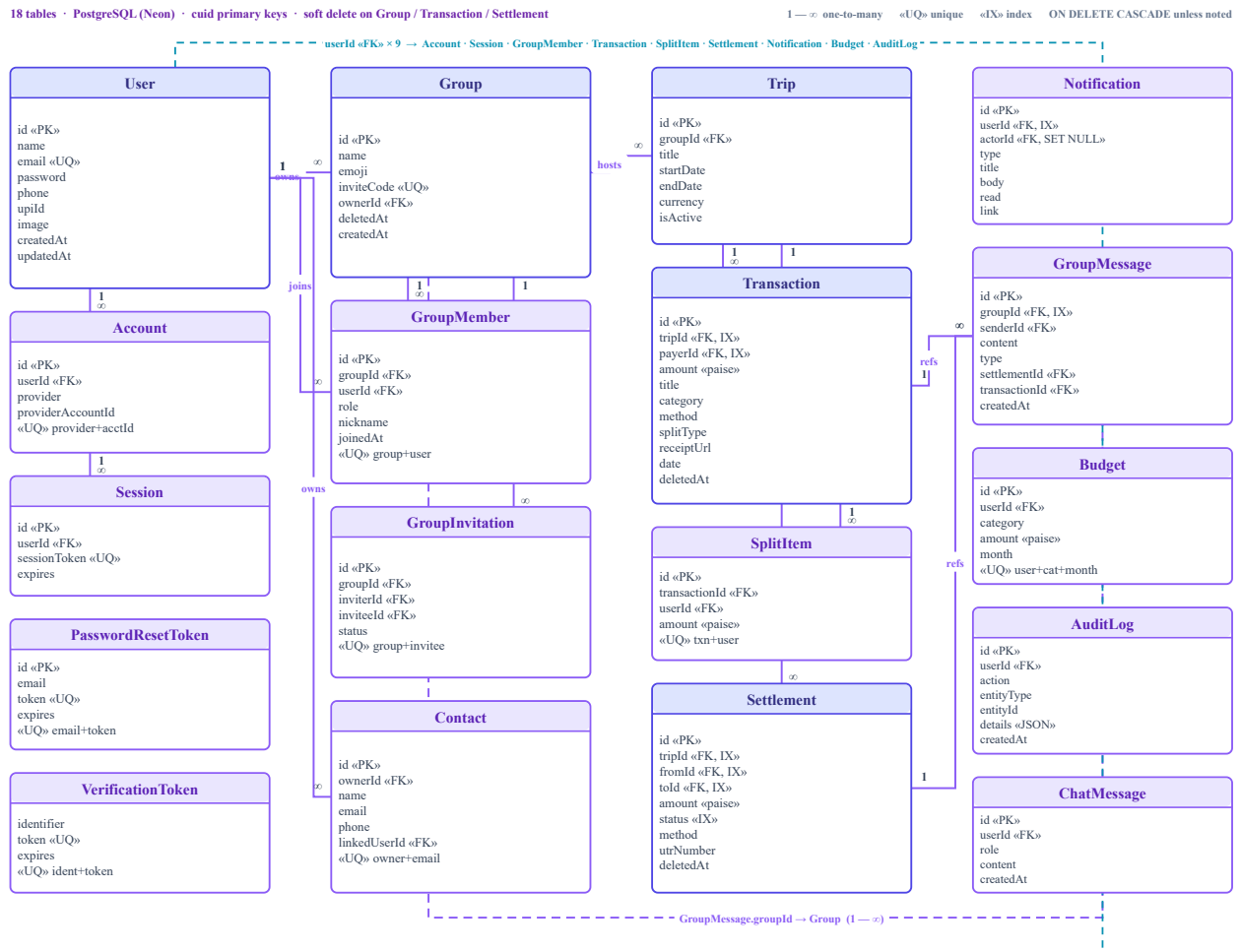


Figure 11 — All eighteen tables with primary keys, foreign keys, unique constraints and indexes; one-to-many relationships are labelled 1 — ∞.

Package Diagram UML STRUCTURAL



Figure 12 — Source packages (indigo), infrastructure-as-code packages (orange) and delivery/observability packages (green), with «import»/«access» dependencies.

Sequence Diagram — Add Expense, Compute Settlement, Pay, Approve UML BEHAVIOURAL

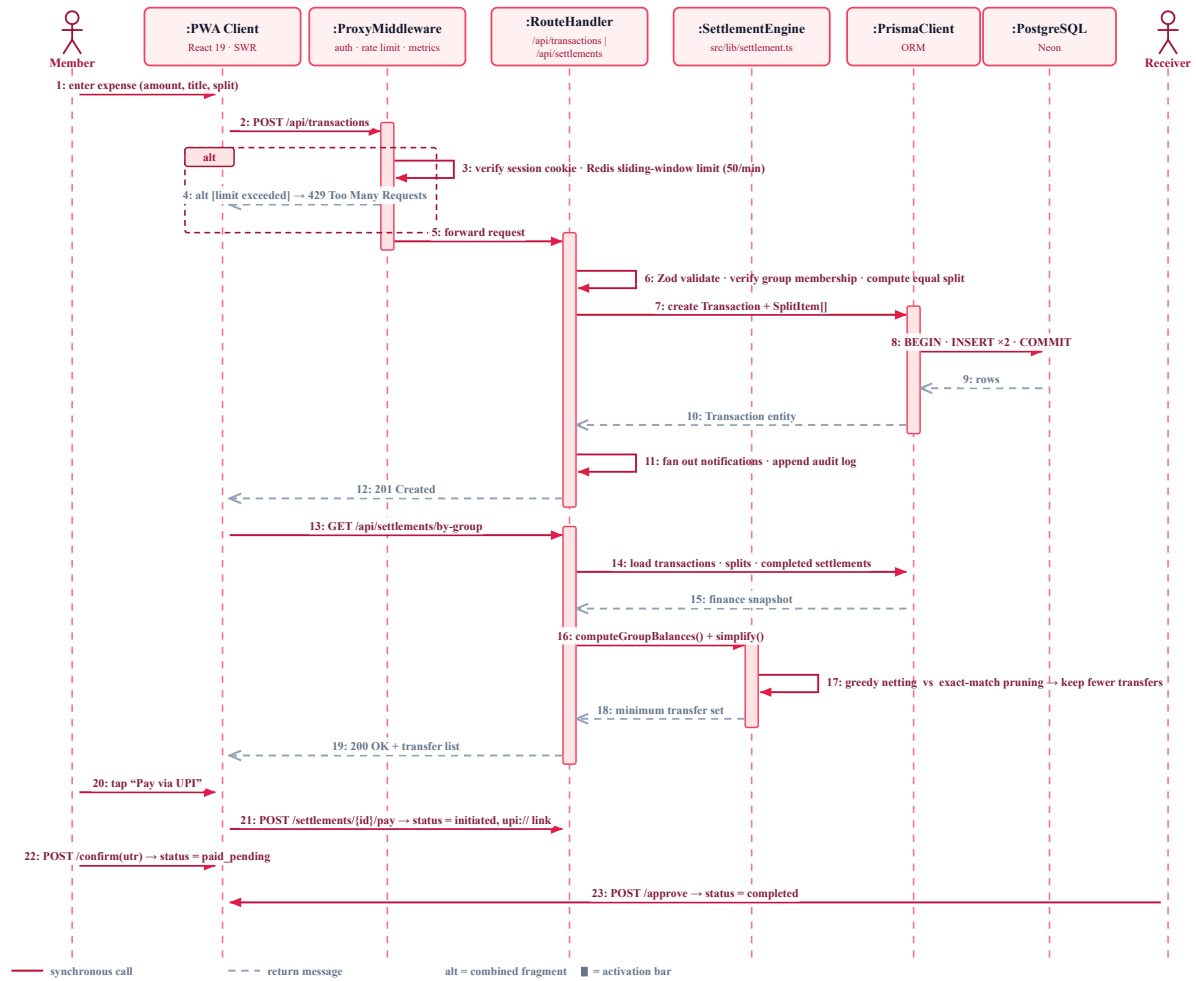


Figure 13 — Twenty-three time-ordered messages across eight lifelines, with activation bars, return messages, self-messages and an alt combined fragment for the rate-limit branch.

State Machine Diagram — Settlement Lifecycle UML BEHAVIOURAL

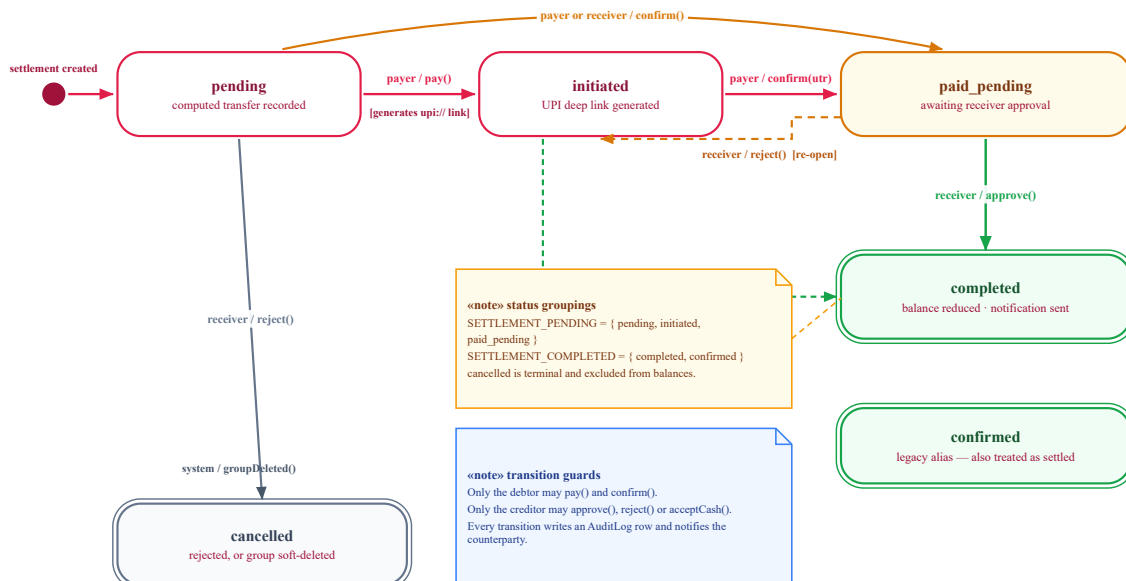


Figure 14 — The six real settlement states with every permitted transition, its triggering actor, and its guard.

Component Diagram UML STRUCTURAL · PROVIDED / REQUIRED INTERFACES

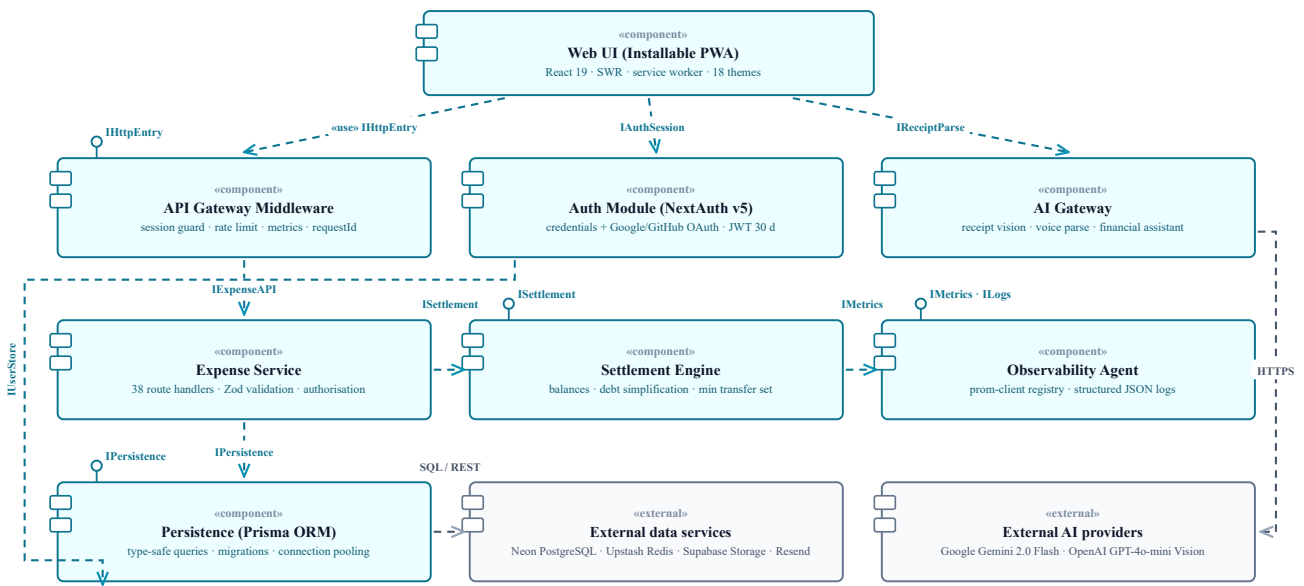
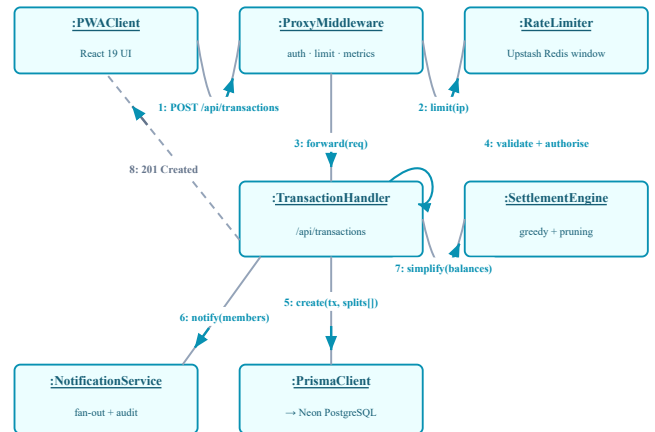
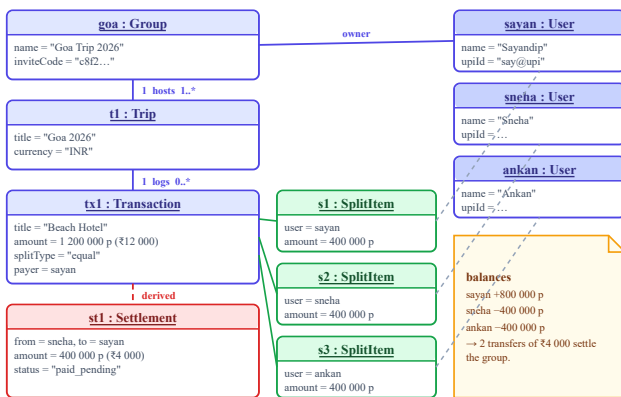


Figure 15 — Components with their provided interfaces (lollipops) and required-interface dependencies.

Object Diagram & Communication Diagram UML INSTANCE AND COLLABORATION VIEWS



sequence numbers on the links express ordering; the same interaction is shown time-ordered on page 9

Figure 16 — Object diagram: a concrete ₹12,000 expense split three ways, and the settlement it produces. Figure 17 — Communication diagram: the same interaction as Figure 13, shown as numbered messages over object links.

Results Achieved

- Settlement reduced from $O(n^2)$ naive transfers to a **minimum transfer set** (typically $n-1$).
- 3-stage image, 430 MB** and non-root, versus ~1.3 GB for an equivalent single-stage build.
- 2 → 10 pods in ~90 s** under a k6 ramp; scale-in after the 120 s stabilisation window.
- 0.00% failed requests** across a rolling update, verified by k6 during helm upgrade.
- 12-stage pipeline with 4 hard gates** (tests, SonarQube, Trivy CRITICAL, k6 smoke) that genuinely fail the build.
- CloudFront serves static assets from cache (**x-cache: Hit**) while `/api/*` correctly bypasses it.
- 9 alert rules** routed through Alertmanager to a webhook within ~60 s of a fault.

Conclusion

SplitX demonstrates that an application and its delivery platform are one system. The same expense-splitting product becomes materially more reliable once it is reproducibly built, declaratively provisioned, automatically scaled and continuously observed.

Every tool in the chain earns its place by answering a concrete question: **Docker** — will it run identically everywhere? **Terraform** — can the infrastructure be rebuilt from nothing? **Kubernetes** — what happens when a pod dies or traffic triples? **Jenkins** and **GitHub Actions** — can a bad commit reach production? **Prometheus**, **Loki** and **Grafana** — would we even notice?

The autoscaling, caching and rollout behaviour reported here is measured from a running cluster, not asserted from configuration.

Future Scope

- KEDA / prometheus-adapter** for event-driven scaling on requests-per-second rather than CPU.
- Argo Rollouts** for canary and blue-green releases with automated metric analysis.
- OpenTelemetry** distributed tracing to join a Loki log line to its exact span.
- Karpenter + Spot** node provisioning and CloudFront multi-region failover.
- External Secrets Operator** backed by AWS Secrets Manager, replacing in-cluster Secrets.
- Service mesh mTLS** and policy enforcement (OPA Gatekeeper) between workloads.
- Multi-currency settlement with FX snapshots, and a native mobile shell.